WorkWebs Audio Mixing Ecosystem Tools Documentation

By Larry T. Jost

Version 20261030a

WorkWebs.com https://github.com/ltjost/WorkWebs-Audio-Mixing-Ecosystem-Tools

Support@WorkWebs.com

https://www.facebook.com/profile.php?id=61557113121231

IMPORTANT:

The tools I have written are available from WorkWebs.com. External components are available as shown below.

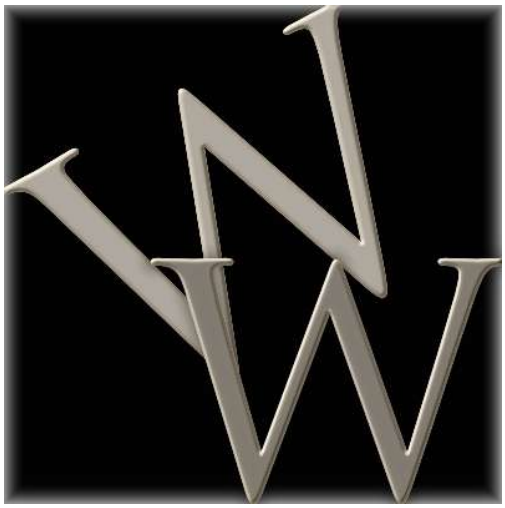
Use REAPER 7.25 or later.

There is no warrantee expressed or implied. Use this at your own risk. I am not responsible for any errors, omissions, or defects in this software solution. Be sure to test everything you use and verify its operation is to your satisfaction.

You may rework any of this as long as you do not require payment for your derived product and as long as you reference WorkWebs.com as the source for the original software.

If you would like to send a Thank You gift you may do so from this link or QR code:

https://www.paypal.com/ncp/payment/7W434P69MWXA8



Contents

Changes made in this release 8

Cue System 9

Overview 9

Setup Files 10

For the BUILD step 10

Build_Device_Types.cmd 11

Build_File_Extension.cmd 11

Build _Variable_From_File.cmd 12

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS WING 14

Build_WING_Track_Values.cmd 14

Build_WING_Colors_Values.cmd 15

Build_WING_DCA_Tag_Values.cmd 16

Build_WING_Connection_Values.cmd 16

Connection Commands 17

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS X32 17

Build_X32_Channel_Values.cmd 17

Build_X32_Bus_Values.cmd 18

Build_X32_DCA_Values.cmd 19

Build_X32_Color_Values.cmd 20

Build_X32_Route_Values.cmd 20

Build_X32_Send_Strings.cmd 22

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS REAPER 23

Build_REAPER_Channel_Values.cmd 23

Build_REAPER_Send_Strings.cmd 23

For the RUN step 25

Run_Device_Info.cmd 25

Run_File_Extension.cmd 25

Run_Cues.cmd 26

CUE FILE CONTENT by Type (WING, X32, REAPER, PASSTHROUGH) 28
PASSTHROUGH CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS 28
CUE FILE Content for PASSTHROUGH Commands 28
VARIABLEFROMFILE CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS 28
CUE FILE Content for VARIABLEFROMFILE Commands 28
WING CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS 30
createfile 30
CUSTOMStartCue 30
CUSTOMEndCue 30
TRACKMUTE 30
TRACKLED 31
TRACKNAME 31
TRACKCOLOR 31
TRACKICON 31
TRACKPAN 31
TRACKPOSTINS 31
TRACK_GATE_ON 31
TRACK_GATE_OFF 31
TRACK_EQ_ON 31
TRACK_ EQ _OFF 32
TRACK_DYN_ON 32
TRACK_ DYN _OFF 32
TRACK_PREINS_ON 32
TRACK_ PREINS _OFF 32
TRACK_POSTINS_ON 32
TRACK_ POSTINS _OFF 32
TRACKPROC 32
TRACKTAG 33
TRACKTAGMUTE 33
TRACKGROUPIN 33

TRACKSEND 33
SENDand_UNMUTE 33
SENDand_MUTE 33
USB_PLAY 33
USB_PLAYFILE 33
USB_RECORD 33
LIVE_RECORDER 34
LOADSCENE 34
NAVSCENE 34
AUTOMIXXON 34
AUTOMIXXOFF 34
AUTOMIXYON 34
AUTOMIXYOFF 34
CUSTOM_a_DCA_ONUNMUTED 35
CUSTOM_a_DCA_OFF 35
Custom_a_CHAN_ON 36
Custom_a_CHAN_OFF 36
SETLATESTDCA 37
SETTRACKTOLATESTDCA 37
SETLATESTMUTEtoMUTE 39
SETLATESTMUTEtoUNMUTE 39
SETTRACKTOLATESTMUTE 39
#| 39
X32 CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS 40
createfile 40
CUSTOMStartCue 40
CUSTOMEndCue 40
CHAN 40
CHANNAME 40

CHANCOLOR 41
CHANSEND 41
CHANDCA 41
DCA 41
DCANAME 41
DCACOLOR 41
BUS 41
BUSNAME 41
BUSCOLOR 42
MAINST 42
MAINSTNAME 42
MAINSTCOLOR 42
MAINM 42
MAINMNAME 42
MAINMCOLOR 42
SENDand_UNMUTE 42
SENDand_MUTE 42
ROUTE_OUT 43
ROUTE_IN 43
USB_RECORDER 43
LIVE_RECORDER 43
SAVESCENE 43
LOADSCENE 43
SAVESNIPPET 43
LOADSNIPPET 43
#| 44
REAPER CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS 45
createfile 45
CUSTOMStartCue 45

CUSTOMEndCue 45
SENDand_UNMUTE 46
SENDand_MUTE 46
SENDand_ezMUTE 46
SENDand_UNMUTE_NAME 46
SENDand_MUTE_NAME 46
TRACKPAN 46
SELECT 46
UNSELECT 47
UNMUTE 47
MUTE 47
MARKERName 47
LASTMARKERName 47
GOTOMarker 47
RECORD 47
PLAY 47
STOP 48
PAUSE 48
FXBYPASS 48
FXACTIVE 48
TRACKNAME 48
TRACKVOLUME 48
ACTIONi 48
ACTIONs 48
ARMON 49
ARMOFF 49
AUTOMIXON 49
AUTOMIXOFF 49
PLUGIN_ADJ 49

FXOPENUI 49
FXCLOSEUI 49
#| 49
VARIABLEFROMFILE Cue Content Generators 50
VARIABLEFROMFILE_40chans_1 version 20261026b Excel Tool 50
WING UTILITY Cue Content Generators to COPY and PASTE into Cue Files 52
UTILITY_outputroutes_1 version 20261030a.xlsx 52
UTILITY_inputroutes_1 version 20261030a.xlsx 53
UTILITY_ inputs_1 version 20261028b.xlsx 53
Other Tooling 54
Mixing Station 128 Head Amp Control for X32/M32 54
WING 40 Channel Solo control 54
AutoHotKey Script 55
Bitfocus Companion and Vicreo Listener 55
Links 56

Changes made in this release

- **October 30 2026**
  - **Updated VARIABLEFROMFILE_40_CHANS_1 is now version 20261026b.**
    - **The spreadsheet no longer updates the channel name color to match the DCA.**
  - **Added UTILITY_outputroutes_1 version 20261030a.xlsx to allow a grid-based setup for output routing for the WING.**
  - **Added UTILITY_inputroutes_1 version 20261030a.xlsx to allow a grid-based setup for input settings for channels for the WING. Main and Alt sources can be selected from a grid and the individual/global and initial source choice are able to be set.**
  - **Added UTILITY_inputs_1 version 20261028b.xlsx to allow naming and color coding and icon setting, etc. for WING Inputs.**
  - **Updated the generic 2 WING snapshots. For W1 this changes channel 37 to be an input from an external LR source. Some reverb changes were also made.**
- **October 26 2026**
  - **Updated VARIABLEFROMFILE_40_CHANS_1 is now version 20261026a.**
    - **Split the names OSC into 2 lines.**
    - **Sample Cue 10 and the .txt file for variable files are updated to match as is this document.**
- **October 16 2026**
  - **Reorganized the repository in github**
  - **Added references to Mouse Without Borders**
  - **Updated the Generic REAPER for 128 track recording/playback.**
  - **New Mixing Station layers - removed button for turning off insert 2...changed naming of AMIX 1 and 2 to X and Y**
  - **Changed Mute Group labels to prefix with a number and then space instead of colon**
  - **WING config reverbs changed to add saturation**
  - **Record Flow revised.**
  - **Passthrough and other compressors made the same in parallel flow paths to prevent latency differences.**
  - **Commented out DELAY in the CUe Build Script.**
  - **Pulled user configurable portions of the BUILD and RUN scripts into separate .cmd files so they can be modified for their content without editing the main code file.**
  - **Added the ability to create WING cue content lines in Excel exported as text files and included in cues via cue lines.**
  - **Provided a sample Excel file for cue development. VARIABLEFROMFILE_40chans_1 version 20261016a.xlsx**
  - **Used the WING OSC node syntax to allow these cues to make major changes in almost no time.**

- **September 17 2026**
- **WING now includes the ability to change track color when muting.**
- **Cue files now include an identifier that defines target IP and Port for each OSC command allowing multiple devices to be impacted by a single cue file (1 or more WINGS + 1 or more X32s + 1 or more REAPERs + 1 or more of anything using OSC via passthrough commands.**
-
    - **WING CUES**
        - **You can now build cues for WING that include these commands (much like SETLATESTDCA and SETTRACKTOLATESTDCA):**
        - **Build2v013.cmd now includes the support for this command set:**
        - **:: SETLATESTMUTEtoMUTE**
        - **:: sets the SETTOLATESTMUTEVALUE to MUTE**
        - **:: SETLATESTMUTEtoUNMUTE**
        - **:: sets the SETTOLATESTMUTEVALUE to UNMUTE**
        - **:: SETTRACKTOLATESTMUTE**
        - **:: sets the specified track to the latest specified SETLATESTMUTE designation**
        - **:: SETTRACKTOLATESTMUTE|myTrackVar's name|**
        - **This could be handy if you want to use the SETLATESTDCA and SETTRACKTOLATESTDCA for major cues that take time to set up and then**
        - **use SETLATESTMUTEtoMUTE, SETLATESTMUTEtoUNMUTE and SETTRACKTOLATESTMUTE**

Cue System

Overview

The CUE system uses Cue files to hold cues. The Cues are text files you write using commands custom designed in this software and can be sent to any device. Each line of the cue file specified a target device. Passthrough raw OSC commands can also be included to send to software you are using even if this system does not have special handling for the desired device (such as MultiPlay for playing audio files). The custom defined commands interpreted in this tool are for Behring WING or X32 or REAPER.

For the WING, there is also the ability to place cue content into cues where that content resides in separate files. These can be created with any desired tool, but included in this release is a sample cue content creation tool that uses Excel.

Excel tools are also provided to create Cue text you can copy/pasts into cues to set up input and output routing and input names and attributes to make doing so easier. The routing for WING in these Excel files is set via a grid view. Just set up what you want and copy the output into the clipboard and paste it inti a cue. Then use the cue to set the values.

The BUILD CMD Script is run to process the CUES and build OSC commands with identifiers for each target device.

The RUN CMD script is used to run the cues built by the BUILD script in real time.

The system uses SENDOSC to send commands. This is available at:

https://github.com/yoggy/sendosc

Remote control of the cue execution can be achieved with Bitfocus Companion with Vicreo Listener, a configuration is provided with this package.

You are also able to skip the build process and just make your own output files and they will still work with the RUN script. This allows you to write OSC and send it to any device you want, either in a sequence or by name as desired without the need to use the BUILD process.

ALL CUES, BUILD, and RUN scripts and the SENDOSC executable and associated installation files, if any, should all be placed the SAME DIRECTORY for this system to operate.

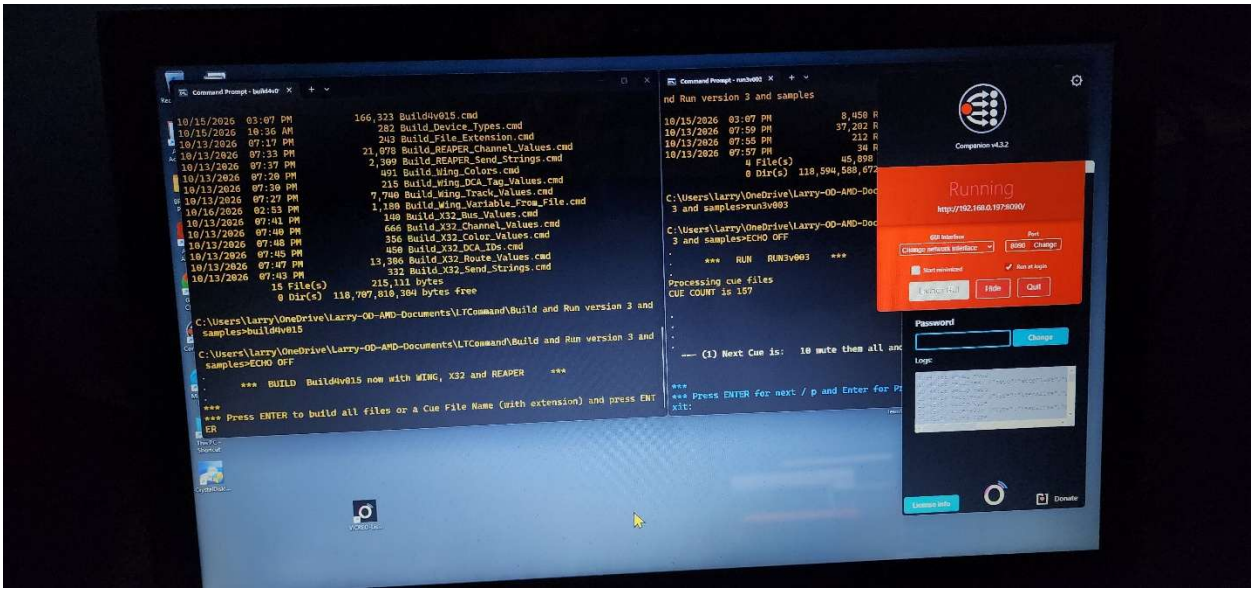


Figure 1Build, Run cmd session windows along with Companion and Vicreo Listener

Setup Files

The RUN and BUILD scripts pull file content from several definition files that define your specific setup. You will need to supply these, but can leverage the samples included with this package.

For the BUILD step

Definition Files for the BUILD process step

NOTE: Names you specify must be unique in that you cannot use the same myChan for a REAPER and WING at the same time, or for 2 WINGS, unless you want them to refer to the same track or channel. It is recommended that you ensure uniqueness by your naming convention.

Build_Device_Types.cmd

Format:

Set Type.xxxxx={WING,REAPER,X32,PASSTHROUGH}

You choose your own value for xxxxxx and this same value needs to be in the Run_Device_Info.cmd file and is also used in each cue line to specify the target device for that cue line.

Example:

set Type.WING1=WING
set Type.WING2=WING
set Type.REAPER1=REAPER
set Type.X1=X32
set Type.PASSTHROUGH=PASSTHROUGH

Explanation:

The text “WING1” or “WING2” or “REAPER1” or “X1” are the names I will use in each line in each cue to specify the target of the line in the cue. These same names are used in the Run_Device_info to provide the additional information the RUN script will need for each of these entities.

The value to the right of the equal sign tells the system which type of device is the target so that the cue information is properly understood and processed. There are only 4 choices: WING, REAPER, X32, and PASSTHROUGH. Note that PASSTHROUGH is not a device per-se, but is an indication that the cue content includes a device name that the RUN script will use.

Build_File_Extension.cmd

Format:

set myFileExtension=xxxxx

You choose your own value for xxxxxx.

Example:

set myFileExtension=cuex

Explanation:

The text cuex is the extension you will be using for your source cue files. The Build process will pull files in the current folder with that extension to build into runnable cues. The file output by build will also include this extension name followed by OUT and the extension name again. The RUN script will read those files.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can by default use Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

For the Current User and setting the sampleext extension to be recognized as text:

reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

Build _Variable_From_File.cmd

Used when the CUE TYPE is WING, REAPER, or X32

Format:

This command contains some batch script code to read the desired file into variables in the BUILD process.

It is used with the VARIABLEFROMFILE cue command to set the cue content as variables that will be defined from an external file's content. In a sense, it is a way to import cue strings from files.

The set of code to be used and repeated with some modification for each external file to use is:

```
for /F "delims=" %%a in ('type VariableCueTextSample.txt') do (
        set "line=%%a"
        if !line!=="" goto :CompletedVariableFile1
::      set "cleaned_line=!line:"=!"
        set "cleaned_line=!line!"
    !cleaned_line!
)
:CompletedVariableFile1
```

In this example,
VariableCueTextSample.txt is the file name to read in that contains the cue variable definitions.
CompletedVariableFile1 is the label to jump to once the input from the file is completed. It appears in the code to tell the script where to jump to and then again is used to label that position in the file.

If you have multiple files to be read, just repeat this whole set of code multiple times as needed but change the name of the input file and increment the number at the end of the jump information. If you change the jump information in this manner it will not conflict with other jump information I the other code that is contained in the BUILD script.

Explanation:

The file is read in and processed to set variables as specified in the file**. See more detail in the description of the VARIABLEFROMFILE cue entry below.**

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS WING

Build_WING_Track_Values.cmd
Used when the CUE TYPE is WING

Format:

set myTrackVar.xxxxx=y
set myTrackVar.xxxxx.type={ch, aux, mtx, bus,main,io,dca,/in/{LCL,AUX,A, etc }, io/out/{LCL,AUX,A, etc } }
set myTrackVar.xxxx.color.on=z
set myTrackVar.xxxx.color.off=w
You choose your own value for xxxxxx.

y is a number that specifies the channel number.

The type value is one value from the set of available choices shown.

The color.on value of z is the color to make the channel if it is unmuted. This value is optional and if not specified a change in a cue to mute or unmute will not change the channel color.

The color.off value of w is the color to make the channel if it is muted. This value is optional and if not specified a change in a cue to mute or unmute will not change the channel color.

Example:

set myTrackVar.w1cCaroline=1
set myTrackVar.w1cCaroline.type=ch
set myTrackVar.w1cCaroline.color.on=%myWingColor.PINK
set myTrackVar.w1cCaroline.color.off=%myWingColor.GRAY

Explanation:

The text myTrackVar. is required. Following the period and preceding the equal sign is a name you are wanting to name this Track. After the equal sign is the trac) number it will reference. A track can have as many names as you would like. If you change the value of a name then the prior value will not be used. Each Track needs a type and these are the strings used in the OSC for the wing. The colors are numbers as defined in WING OSC. But you can use the variables specified for colors in Build_Wing_Colors as well as shown I the example since they are set before this value is referenced. With this naming in place you can make more readable cues and can change definitions if needed and then rebuild the cues without editing each cue file.

The values are used as follows in building the OSC:

/type/y/col

So if Y is 1 and TYPE is /io/in/LCL and color is 2 then the OSC will be:

/io/in/LCL/1/col ,i 2

Please reference the WING REMOTE PROTOCOLS document for all possible OSC uses for this format of definition. The build OSC just takes the values you provide and strings them together and adds the "col" to make it a color value setting.

Build_WING_Colors_Values.cmd

Used when the CUE TYPE is WING

Format:

set myWingColor.xxxxx=y

You choose your own value for xxxxxx. y is a number that specifies the color number you are defining. *Please reference the WING REMOTE PROTOCOLS document for details.*

Example:

set myWingColor.MEDIUMBLUE=1
set myWingColor.BRIGHTBLUE=2
set myWingColor.VIOLET=3
set myWingColor.TURQUOISE=4
set myWingColor.GREEN=5
set myWingColor.OLIVEGREEN=6
set myWingColor.YELLOW=7
set myWingColor.BROWN=8
set myWingColor.RED=9
set myWingColor.CORAL=10
set myWingColor.PINK=11
set myWingColor.DARKVIOLET=12
set myWingColor.GOLD=13
set myWingColor.LIGHTBLUE=14
set myWingColor.TANGERINE=15
set myWingColor.LIGHTGREEN=16
set myWingColor.GRAY=17
set myWingColor.WHITE=18

Explanation:

The name specified, such as GREEN, can be used in cues to specify color 5, etc.

Build_WING_DCA_Tag_Values.cmd

Used when the CUE TYPE is WING

Format:

set myTAGVar.xxxxx=y

You choose your own value for xxxxxx. Y can be “#D1” through “#D16” as defined by the WING protcols to specify DCA 1 through 16 or can be any combination of DCA values if separated by a comma. For example “#D1,#D2” would tie the name you specified to placing a channel into both DCA1 and DCA2.

Example:

```
set   myTAGVar.w1dcaOTHER="#D1"
set   myTAGVar.w1dcaENS1="#D2"
set   myTAGVar.w1dcaENS2="#D3"
set   myTAGVar.w1dcaENS3="#D4"
set   myTAGVar.w1dcaENS4="#D5"
set   myTAGVar.w1dcaOneandTWO="#D1,#D2"
```

Explanation:

The name specified, such as w1dcaOTHER, can be used in cues to specify the TAG #D1

Build_WING_Connection_Values.cmd

Used when the CUE TYPE is WING

set myCONNVar.xxxxx.type=y

set myCONNVar.xxxxx.num=z

The WING channels need to have connections set in place to indicate the source for the channel. This is done by a Group designation and a number designation within the group.

See the WING OSC manual for details.

Xxxxx is a name you desire to use for the connection.

Y is a type (group) and a Channel connection can be from any of these groups

OFF, LCL, AUX, A, B, C, SC, USB, CRD, MOD, PLAY, AES, USR, OSC, BUS, MAIN, MTX

Z is a number, such as a number from 1-64 to designate the number of the connection of the specified type.

Example:

set myCONNVar.JoeMic.type=B
set myCONNVar.JoeMic.num=1

Explanation:

The name specified, such as Joe Mic, can be used in cues to the connection for AES50 B (type) 1

Connection Commands

The WING channels need to have connections set in place to indicate the source for the channel. This is done by a Group designation and a number designation within the group. See the WING OSC manual for details, but for example, a Channel connection can be from any of these groups: OFF, LCL, AUX, A, B, C, SC, USB, CRD, MOD, PLAY, AES, USR, OSC, BUS, MAIN, MTX with a number from 1-64. Likewise for an AUX.

In the BUILD script you may say:

set myCONNVar.micSEN1.type=B
set myCONNVar.micSEN1.num=1

to indicate that a reference to micSEN1 refers to AES50 B 1.

In a CUE you could use:

TRACKGROUPIN|cJenny|micSEN1|
to indicate that the channel named cJenny is to be connected to whatever micSEN1 indicated, namely AES50 B 1.

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS X32

Build_X32_Channel_Values.cmd

Used when the CUE TYPE is X32

Format:

set myChan.xxxxx=yy

You choose your own value for xxxxxx. yy is a 2-digit number that specifies the (channel) number you are defining. Note for the X32 this must be 2 digits so for 1 use 01, etc.

Example:

set myChan.cVIOLIN1=28
set myChan.cVIOLIN2=29
set myChan.cVIOLA=27
set myChan.cCELLO=31
set myChan.cKEYS=01
set myChan.cAG=04

Explanation:

The text myChan. is required. Following the period and preceding the equal sign is a name you are wanting to name this channel. After the equal sign is the channel number it will reference. A channel (track) can have as many names as you would like. If you change the value of a name then the prior value will not be used. With this naming in place you can make more readable cues and can change definitions if needed and then rebuild the cues without editing each cue file.

Build_X32_Bus_Values.cmd
Used when the CUE TYPE is X32

Format:

set myBus.xxxxx=yy

You choose your own value for xxxxxx. yy is a 2-digit number that specifies the bus you are defining. Note for the X32 this must be 2 digits so for 1 use 01, etc.

Example:

set myBus.testbus1=01

Explanation:

The text myBus. is required. Following the period and preceding the equal sign is a name you are wanting to name this bus. After the equal sign is the bus number it will reference. A bus can have as many names as you would like. If you change the value of a name then the prior value will not be used. With this naming in place you can make more readable cues and can change definitions if needed and then rebuild the cues without editing each cue file.

Build_X32_DCA_Values.cmd

Used when the CUE TYPE is X32

There are 2 types of definitions specified in this file. One for single DCA use and one for multiple DCA use.

SINGLE DCA use:

Format:

set myDCA.xxxxx={1,2,3,4,5,6,7,8}

You choose your own value for xxxxxx. a value from 1-8 to specify the DCA.

Example:

set myDCA.testdca1=1

Explanation:

You can use testdca1 to refer to DCA 1 in single DCA usage in cues.

MULTIPLE DCA use:

Format:

set myDCAasssign.xxxxx=y

You choose your own value for xxxxxx. y value from 0-255 to specify the DCA combination to use. Each DCA has a bit for its assignment as follows and these values are summed.

0 being no DCAs to 255 meaning All DCAs on.
DCA 1 is 1
DCA 2 is 2
DCA 3 is 4
DCA 4 is 8
DCA 5 is 16
DCA 6 is 32
DCA 7 is 64
DCA 8 is 128
DCA 1 and 3 is 1+4=5
etc.

Example:

set myDCAassign.testassigneddca1=1

Explanation:

You can use testassigneddca1 to refer to DCA 1 in multiple DCA usage in cues.

Build_X32_Color_Values.cmd
Used when the CUE TYPE is X32

Format:

set myColor.xxxxx=y

You choose your own value for xxxxxx. y is a color number that specifies the color to use.

Example:

set myColor.OFF=0
set myColor.RD=1
set myColor.GN=2
set myColor.YE=3
set myColor.BL=4
set myColor.MG=5
set myColor.CY=6
set myColor.WH=7
set myColor.OFFi=8
set myColor.RDi=9
set myColor.GNi=10
set myColor.YEi=11
set myColor.BLi=12
set MyColor.MGi=13
set myColor.CYi=14
set myColor.WHi=15

Explanation:

The name specified, such as RD, can be used in cues to specify color 1, etc.

Build_X32_Route_Values.cmd
Used when the CUE TYPE is X32

Format:

set myRouteIn.xxxxx=yy

or

set myRouteOut.xxxxx=yy

These are the User In and User Out numbers and must be 2 digits zerofilled.

PLUS define the pre-defined route numbers in this format:

set myRouteSourceIn.xxxxx=y

or

set myRouteSourceOut.xxxxx=y

You choose your own value for xxxxxx. y is the number to use between 0 and 208. These are predefined 0-168 for In Routing and 0-208 for Out routing. Reasonable prebuilt definitions are in the sample file and explain the choices clearly for Local, AES50A, AES50B, CARD, AUX, OUT, P16, Mon and Talkback settings. These are not further described here but are defined in the sample file for your use.

Examples for myRouteIn and my RouteOut:

myRouteOut.Cello=01

Explanation:

Here the route named Cello is User Out 1

Be sure to keep a record of the mapping to numbers in the x32 definitions for user out routing:

- 0 is OFF
- 1-32 are Local 1-32
- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6
- 167 is TalkBack Internal
- 168 is Talkback External
- 169-184 are OUIT 1-16
- 185-200 are P16 1-16
- 201-206 are AUX 1-6
- 207 is Main L
- 208 is Main R

Be sure to keep a record of the mapping to numbers in the x32 definitions that can be used for user in routing:

- 0 is OFF
- 1-32 are Local 1-32
- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6

- 167 is TalkBack Internal
- 168 is Talkback External

Build_X32_Send_Strings.cmd

Used when the CUE TYPE is X32

Format:

set mySend.xxxxx=string

xxxxx is the name you choose for this send string. The send string specifies what to do with each of the 16 x32 sends for a channel based on string positions 1-16:

1 sets the send to not muted
0 sets the send to muted
x does not change the send
- does not count in the increment of the send number and is ignored
* terminates the processing of the string

Example:

set mySend.solo=1^00-00*

Explanation:

A send name of solo is going to cause send 1 to be not muted and send 2 to be unchanged and sends 3-6 to be unmuted.

THESE ARE USED FOR CUES WHERE THE BUILD DEVICE TYPE SPECIFIED IN THE CUE IS REAPER

Build_REAPER_Channel_Values.cmd

Used when the CUE TYPE is REAPER

Format:

set myChan.xxxxx=y

You choose your own value for xxxxxx. y is a number that specifies the track (channel) number you are defining.

Example:

set myChan.r1Zara=9
set myChan.r1Naomi=10
set myChan.Chan9=9
set myChan.Chan10=10

Explanation:

A channel is a track in REAPER but the code was written to use “Chan”. The text myChan. is required. Following the period and preceding the equal sign is a name you are wanting to name this channel. After the equal sign is the channel (track) number it will reference. A channel (track) can have as many names as you would like. If you change the value of a name then the prior value will not be used. With this example, you can specify “r1Zara” to specify channel 9 or you can use “Chan9” to refer to that same channel (track). With this naming in place you can make more readable cues and can change definitions if needed and then rebuild the cues without editing each cue file.

Build_REAPER_Send_Strings.cmd

Used when the CUE TYPE is REAPER

Format:

set mySend.xxxxx=string

xxxxx is the name you choose for this send string. The send string specifies what to do with each of the sends for a channel based on string positions 1-n:

1 sets the send to the cxue-specified volume level

0 sets the send to 0 volume

x does not change the send

- does not count in the increment of the send number and is ignored

* terminates the processing of the string

Example:

set mySend.slines1=100000-00-0*

Explanation:

A send name of slines1 is going to cause send 1 (send to BUS1 perhaps is defined I nREAPER) to be not muted and sends 3-9 to be unmuted.

For the RUN step

Run_Device_Info.cmd

Format:

Set IP.xxxxx=yyy.yyy.yyy.yyy

Set Port.xxxxx=z

You choose your own value for xxxxxx and this same value needs to be in the Build_Device_Type.cmd file and is also used in each cue line to specify the target device for that cue line. Here y… is the IP of the device specified and z is the port of the device specified to send the OSC commands to that reference that name.

Example:

set IP.WING1=192.168.0.11
set Port.WING1=2223
set IP.WING2=192.168.0.12
set Port.WING2=2223
set IP.REAPER1=192.168.0.107
set Port.REAPER1=8007
set IP.MULTIPLAY1=192.168.0.107
set Port.MULTIPLAY1=8888

Explanation:

The text “WING1” or “WING2” or “REAPER1” or “MULTIPLAY1” are the names to use in each line in each cue to specify the target of the line in the cue. These same names are used in the Build_Device_Types.cmd file to provide the additional information the BUILD script will need for each of these entities.

The value to the right of the equal sign tells the system which IP and Port to use for each device.

Run_File_Extension.cmd

Format:

set myFileExtension=xxxxx

You choose your own value for xxxxxx.

Example:

set myFileExtension=cuex.OUTcuex

Explanation:

The text cuex.OUTcuex is the extension the BUILD used for writing your cue files.

The RUN script will use this extension for cue content file reading.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can by default use Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

For the Current User and setting the sampleext extension to be recognized as text:

reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

Run_Cues.cmd

Format by example:

You could hard code the cue numbers, but since they run in sequence if used that way they should be listed in the file in sequence and assigned sequential numbers. Since references in cues use the file name, the file numbers only establish the sequential order.

Here is the recommended way to do this. Place this sequence of lines in the file with 1 set per cue:

```
set /a cuecount=0

set /a cuecount+=1
set cue[%cuecount%].cuename=10
set cue[%cuecount%].file=10
set cue[%cuecount%].cuedescr=mute them all and all out of DCAs
set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call

set /a cuecount+=1
set cue[%cuecount%].cuename=20
set cue[%cuecount%].file=20
set cue[%cuecount%].cuedescr=all in their dcas BUT DCAs Muted
```

set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call

Explanation:

Cues are created with an ordered sequence, which you can use the sequential aspect of them or not use it, but the order is there. You run the RUN script to process the cues. You can simply enter the "cuename" to run the desired cue. To run them in sequence, you would just hit enter to go to the next cue. (You can back-up by entering "p".)

You can use a Cue File in more than one cue, but each cue needs a unique cuename value.

The cuedescr value is displayed in the command prompt window when the cue is run. You can wait a while before running a cue. The number of seconds to wait is set in the afterpause variable for the cue. If it is omitted then 0 is used.

If you provide a ue name assignment to the follow variable then the specified cue will run after the current cue is processed without you taking any further action.

The last thing in each cue definition is an indication of how to process the cue. The choices are "call" or "start". If you specify "call" then the RUN script will process the cue and wait until it completes before allowing further action. If you specify "start" then an additional command prompt window is opened and runs the cue and the RUN script is ready for additional processing as soon as the cue is started. THIS IS NOT RECOMMENDED IF THE CUE has more than a couple commands in it as it drives a lot of overhead and process starting.

There are lots of design choices to make when writing cues. Keep in mind that a lot of commands are available and can result in many OSC commands. If you anticipate running sequential cues out of sequence (such as in theater if actors skip a chunk of the show and come back or in rehearsal when you jump around, then each cue needs to get the entire system in a suitable state. If this involves setting 30 track mutes, 30 track namings, and other things the cue may take longer than desired to execute.

Note that you can run multiple RUN scripts on one or more networked computers. You may want to do this to aid in operational effectiveness or you may want to run sequential cues plus have non-sequential "commands" as cues in another setting. This lets you use utility cues at any time while not interrupting sequential flows. Using AutoHotKey (or similarly Bitfocus Companion and Vicreo listener for remote web based access) you can set up keyboard keys as single press commands for any of the RUN script. This is a great way to build a custom keyboard to process custom commands that are using OSC control!

CUE FILE CONTENT by Type (WING, X32, REAPER, PASSTHROUGH)

PASSTHROUGH CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS

CUE FILE Content for PASSTHROUGH Commands

A passthrough command is added to a cue file as follows. This example passes /cue/10/go to the device defined as MUTIPLAY1 (in Build_Device_Types.cmd and Run.Device.Info)

PASSTHROUGH|MULTIPLAY1^^/cue/10/go

VARIABLEFROMFILE CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS

CUE FILE Content for VARIABLEFROMFILE Commands

VARIABLEFROMFILE is used to generate cue content external to this system and insert it into CUE files. The VARIABLEFROMFILE cue command is available for all 3 device types (REAPER,X32,WING).

When using VARIABLEFROMFILE the cues must be developed externally and the naming and reference names you defined are not used. Everything is hard coded externally and simply imported.

Let's look at an example.

In Build_Variable_From_File.cmd is the code to read from a file.

In that file are text lines that were built from outside of this system that look like this one line. The BUILD script reads the file specified and runs the lines of code that are present in the file, such as shown below to set values into the variables in that line of code:

```
set w1_0001_ch_col_mute=/ s
/ch.1.col=13,mute=0,.2.col=13,mute=0,.3.col=13,mute=0,.4.col=13,mute=0,.5.col=13,mute=0,.6.col=13,mute=0,.7.col
=13,mute=0,.8.col=13,mute=0,.9.col=13,mute=0,.10.col=10,mute=0,.11.col=10,mute=0,.12.col=8,mute=0,.13.col=11,
mute=0,.14.col=11,mute=0,.15.col=11,mute=0,.16.col=11,mute=0,.17.col=11,mute=0,.18.col=11,mute=0,.19.col=5,mu
te=0,.20.col=5,mute=0,.21.col=5,mute=0,.22.col=4,mute=0,.23.col=4,mute=0,.24.col=17,mute=1,.25.col=8,mute=0,.26
.col=8,mute=0,.27.col=8,mute=0,.28.col=8,mute=0,.29.col=8,mute=0,.30.col=8,mute=0,.31.col=8,mute=0,.32.col=2,m
ute=0,.33.col=2,mute=0,.34.col=2,mute=0,.35.col=3,mute=0,.36.col=3,mute=0,.37.col=17,mute=1,.38.col=18,mute=0,.
39.col=18,mute=0,.40.col=18,mute=0,
```

In Build the variable w1_0001_ch_col_mute gets set!

When BUILD reads the following CUE file it encounters the variable name in the VARIABLEFROMFILE command (along with others):

```
WING1|CUSTOMStartCue|"Cue 10 Running"|
REAPER1|ACTIONi|40157|
REAPER1|LASTMARKERName||"Cue 10"|

WING1|VARIABLEFROMFILE|w1_0001_ch_col_mute|
WING1|VARIABLEFROMFILE|w1_0001_ch_tags|
WING1|VARIABLEFROMFILE|w1_0001_ch_names_a|
WING1|VARIABLEFROMFILE|w1_0001_ch_names_b|
WING1|VARIABLEFROMFILE|w1_0001_ch_postins_a|
WING1|VARIABLEFROMFILE|w1_0001_ch_postins_b|
WING1|VARIABLEFROMFILE|w1_0001_dcas_amix|


WING1|CUSTOMEndCue|"Cue 10 Done"|
```

When BUILD encounters this Cue content it places the content of the variable specified (w1_0001_ch_col_mute) into the output cue so the OSC present is sent to the device WING1 defines (IP and Port) so the target system is acted upon.

In this example, for this one line of Cue, the following gets written to the cue's OUT file for the RUN script to process (1 line of the file):

```
WING1^/ s
/ch.1.col=13,mute=0,.2.col=13,mute=0,.3.col=13,mute=0,.4.col=13,mute=0,.5.col=13,mute=0,.6.col=13,m
ute=0,.7.col=13,mute=0,.8.col=13,mute=0,.9.col=13,mute=0,.10.col=10,mute=0,.11.col=10,mute=0,.12.col
=8,mute=0,.13.col=11,mute=0,.14.col=11,mute=0,.15.col=11,mute=0,.16.col=11,mute=0,.17.col=11,mute=
0,.18.col=11,mute=0,.19.col=5,mute=0,.20.col=5,mute=0,.21.col=5,mute=0,.22.col=4,mute=0,.23.col=4,m
ute=0,.24.col=17,mute=1,.25.col=8,mute=0,.26.col=8,mute=0,.27.col=8,mute=0,.28.col=8,mute=0,.29.col=
8,mute=0,.30.col=8,mute=0,.31.col=8,mute=0,.32.col=2,mute=0,.33.col=2,mute=0,.34.col=2,mute=0,.35.c
ol=3,mute=0,.36.col=3,mute=0,.37.col=17,mute=1,.38.col=18,mute=0,.39.col=18,mute=0,.40.col=18,mute
=0,
```

Various external code will require specific variables to be specified as produced by the external system.

Such external tools provided with this package are described later in this document.

WING CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS

NOTE: Values designated as “myXxx value” are defined in the BUILD files and described above.

NOTE: THIS PORTION OF THE DOUCMENT INDICATES THE CUE COMMAND THAT MUST STILL BE PREFIXED BY THE DEVICE NAME AS SET IN THE BUILD_DEVICE_TYPES.CMD file.

EACH COMMAND MUST BE PRECEEDED BY devicename|
For example:
WING1|createfile|

If track on and off colors are defined in the Build_Wing_track_values.cmd file then they will be used.,

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it calls createfile to open a new output cue file based on the name of the input cue file
2) it renames the matrix 8 channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value
Format:
CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames Matrix 8 Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed
Format:
CUSTOMEndCue|text|

TRACKMUTE

TRACKMUTE is used to mute or unmute a track
Format:
TRACKMUTE|myTrackVar’s name|MUTE or UNMUTE|

TRACKLED

TRACKMLED is used to turn the track LED on or off
Format:
TRACKLED|myTrackVar's name|ON or OFF|

TRACKNAME

TRACKNAME is used to set a track name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
TRACKNAME| myTrackVar's name |text|

TRACKCOLOR

TRACKCOLOR is used to set the color for a track in the scribble strip. The Color Names are set in the BUILD script (sample: LIGHTBLUE, MEDIUMBLUE, DARKBLUE, TURQUOISE, GREEN, OLIVEGREEN, YELLOW, ORANGE, RED, CORAL, PINK, MAUVE).
Format:
TRACKCOLOR| myTrackVar's name |Color Name|

TRACKICON

TRACKICON is used to set a track icon in the scribble
Format:
TRACKICON| myTrackVar's name |icon number|

TRACKPAN

TRACKPAN is used to set a pan a track. Values are -100 to 100
Format:
TRACKPAN| myTrackVar's name |number|

TRACKPOSTINS

TRACKPOSTINS is used to assign a post insert for a track.
Format:
TRACKPOSTINS| myTrackVar's name |FX or AUTO_X or AUTO_Y|

TRACK_GATE_ON

TRACK_GATE_ON is used turn the GATEeffect on for a track.
Format:
TRACK_GATE_ON| myTrackVar's name |

TRACK_GATE_OFF

TRACK_GATE_OFF is used turn the GATE effect off for a track.
Format:
TRACK_GATE_OFF| myTrackVar's name |

TRACK_EQ_ON

TRACK_ EQ _ON is used turn the EQ effect on for a track.

Format:
TRACK_ EQ _ON| myTrackVar's name |

TRACK_ EQ _OFF
TRACK_ EQ _OFF is used turn the EQ effect off for a track.
Format:
TRACK_ EQ _OFF| myTrackVar's name |

TRACK_DYN_ON
TRACK_ DYN _ON is used turn the DYN effect on for a track.
Format:
TRACK_ DYN _ON| myTrackVar's name |

TRACK_ DYN _OFF
TRACK_ DYN _OFF is used turn the DYN effect off for a track.
Format:
TRACK_ DYN _OFF| myTrackVar's name |

TRACK_PREINS_ON
TRACK_ PREINS _ON is used turn the PREINS effect on for a track.
Format:
TRACK_ PREINS _ON| myTrackVar's name |

TRACK_ PREINS _OFF
TRACK_ PREINS _OFF is used turn the PREINS effect off for a track.
Format:
TRACK_ PREINS _OFF| myTrackVar's name |

TRACK_POSTINS_ON
TRACK_ POSTINS _ON is used turn the POSTINS effect on for a track.
Format:
TRACK_ POSTINS _ON| myTrackVar's name |

TRACK_ POSTINS _OFF
TRACK_ POSTINS _OFF is used turn the POSTINS effect off for a track.
Format:
TRACK_ POSTINS _OFF| myTrackVar's name |

TRACKPROC
TRACKPROC is used to assign a set the GEDI in the desired order for processing. See the OSC manual for options.
GEDI, GEID, GIED, IGED…
Format:
TRACKPROC| myTrackVar's name|text|

TRACKTAG

TRACKTAG is used to assign a TAG value to a track.
Format:
TRACKTAG| myTrackVar's name|myTagVar's name|

TRACKTAGMUTE

TRACKTAGMUTE is used to assign a TAG value to a track and mute or unmute the track
Format:
TRACKTAGMUTE| myTrackVar's name|myTagVar's name|MUTE or UNMUTE|

TRACKGROUPIN

TRACKGROUPIN is used to set a GROUP connection name and number. It is best to do this while the track is muted.
Format:
TRACKGROUPIN| myTrackVar's name|myCONNVar's name|

TRACKSEND

TRACKSEND is used to set the enable or disable a send from a specified track.
Format:
TRACKSEND| myTrackVar's name | mySend's name|MUTE or UNMUTE

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS and then UNMUTE the track.
Format:
SENDand_UNMUTE| myTrackVar's name | mySend's name |

SENDand_MUTE

SENDand_MUTE is used to update SENDS and then MUTE the track.
Format:
SENDand_MUTE| myTrackVar's name | mySend's name |

USB_PLAY

USB_PLAY is used to control playing for the USB recorder
Format:
USB_PLAY|STOP or PAUSE or NEXT or PREV or PLAYFILE|

USB_PLAYFILE

USB_PLAYFILE is used to set the file name to play for when the PLAYFILE is sent with USB_PLAY
Format:
USB_PLAYFILE|full file name|

USB_RECORD

USB_RECORD is used to control recording for the USB recorder

Format:
USB_RECORD|STOP or REC or PAUSE or NEXT or NEWFILE|

LIVE_RECORDER

LIVE_RECORDER is used control the LIVE Card Recorder
Format:
LIVE_RECORDER|1 or 2|STOP or PLAY or PPAUSE or RECORD|

LOADSCENE

LOADSCENE is used to load a scene from its Tag Number
Format:
LOADSCENE|Scene Tag Number|

NAVSCENE

LOADSCENE is used to load a scene
Format:
NAVSCENE|GOPREV or GONEXT or GO or PREV or NEXT|

AUTOMIXXON

AUTOMIXXOFF

AUTOMIXYON

AUTOMIXYOFF

These are used to turn the specified AUTOMIX on or off.

Format:

AUTOMIXXON

Or

AUTOMIXXOFF

Or

AUTOMIXYON

Or

AUTOMXYOFF

CUSTOM_a_DCA_ONMUTED

Custom_a_DCA_ONMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED off (check the code as this may be commented out)
3. It mutes the track
4. In the current code is code to set color YELLOW but this is presently commented out

Format:
Custom_a_DCA_ONMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_ONUNMUTED

Custom_a_DCA_ONUNMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED on (check the code as this may be commented out)
3. It unmutes the track
4. In the current code is code to set color YELLOW but this is presently commented out

Format:
Custom_a_DCA_ONUNMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_OFF

Custom_a_DCA_OFF is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to "."
2. It turns the Track's LED off (check the code as this may be commented out)
3. It mutes the track
4. In the current code is code to sets color DARKBLUE but this is presently commented out

Since text is ignored you can just change the command from a text matching
the Custom_a_DCA_ONMUTED or Custom_a_DCA_ONUNMUTED commands
Format:
Custom_a_DCA_OFF|MyTrackVar's Name|text{but text is ignored}|

Custom_a_CHAN_ON

Custom_a_CHAN_ON is a custom code area used to set up a channel track possibly with AUTO MIX and with a DCA assignment

1. It sets the Track's Post Insert to the specified text (like FX or AUTO_X or AUTO_Y)
2. It turns the Track's Post Insert On
3. It sets the Track's LED on (check the code as this may be commented out)
4. It turns the Track's Post Insert On
5. It sets the Track's DCA tags to match the specified DCA Variable content
6. It runs the tracks through the SENDand_UNMUTE sequence based on the Send String content
7. It colors the track as indicated in COLOR
8. It names the track as indicated in TEXT
9. It unmutes the track

Format:
Custom_a_CHAN_ON|MyTrackVar's Name|MyDCAVar's Name|post insert text|MySendString name|COLOR|TEXT|

Custom_a_CHAN_OFF

Custom_a_CHAN_OFF is a custom code area used to set up a channel track possibly with an "off" status...meaning:

1. It sets the Track's Post Insert to FX
2. It turns the Track's Post Insert On
3. It sets the Track's LED off (check the code as this may be commented out)
4. It sets the Track's DCA tags to match the "dcapark" DCA Variable content
5. It runs the tracks through the SENDand_MUTE sequence for "spark" including that it mutes the track although LED is off, COLOR is set to dark blue
6. It names the track as indicated in TEXT

The inclusion of and non-use of parameters makes it easier to keep editing cues and toggling between these 2 custom commands.

Format:
Custom_a_CHAN_OFF|MyTrackVar's Name|MyDCAVar's Name{ignored}|post insert text {ignored}|MySendString name{ignored}|COLOR{ignored}|TEXT|

SETLATESTDCA

SETLATESTDCA sets the latest DCA to the specified DCA

Format:
SETLATESTDCA|TagVar's name or xxx for not having one|myTrackVar for the DCA|Name to set on the DCA Track

SETTRACKTOLATESTDCA

SETTRACKTOLATESTDCA sets the specified Track to have the Tag from the SETLATESTDCA value

- It sets the tags according to the TagVar in SETLATESTDCA and
- If the Tag content is xxx then it mutes the track and turns off the LED (and removes all tags)
- otherwise, it does set the tags according to the TagVar in SETLATESTDCA and turns on the LED and Unmutes the specified track

Format:
SETTRACKTOLATESTDCA|myTrackVar's name|

These 2 commands allow you to just place a SETLATESTDCA command in a CUE and then follow it by the SETTRACKTOLATESTDCA commands for all tracks you want in that DCA.

If you use xxx in the SETTRACKTOLATESTDCA name then whatever tracks that follow will be muted.

With these you can cut and paste SETTRACKTOLATESTDCA text lines under the desired DCAs to manage the cues. This is very efficient and easy to manage.

NOTE: Track color may be changed based on the on and off color values for the channel.
Here is sample CUE file to demonstrate this:

```
WING1|CUSTOMStartCue|"Cue 140 Running"|
REAPER1|ACTIONi|401574
REAPER1|LASTMARKERName||"Cue 140"|

WING1|#| -------- overall auto mix - everyone is in X excluding ENS-only ----------------------

WING1|AUTOMIXXOFF|

WING1|SETLATESTDCA|w1dcaOTHER|w1dOTHER|" ariel plt grims"|
WING1|SETTRACKTOLATESTDCA  | w1cCaroline  | # Ariel
WING1|SETTRACKTOLATESTDCA  | w1cKeith     | # Grimsby
WING1|SETTRACKTOLATESTDCA  | w1cSyr       | # Pilot



WING1|SETLATESTDCA|w1dcaens1|w1dENS1|"."|
WING1|SETLATESTDCA|w1dcaens2|w1dENS2|"."|
WING1|SETLATESTDCA|w1dcaens3|w1dENS3|"."|
WING1|SETLATESTDCA|w1dcaens4|w1dENS4|"7sailors"|
WING1|SETTRACKTOLATESTDCA  | w1cDavid     | # Sailor Chef
WING1|SETTRACKTOLATESTDCA  | w1cRobert    | # Sailor Chef
WING1|SETTRACKTOLATESTDCA  | w1cThomas    | # Sailor Chef
WING1|SETTRACKTOLATESTDCA  | w1cBill      | # Sailor Chef
```

WING1|SETTRACKTOLATESTDCA | w1cDennis | # Sailor Chef -CHef Louis US-
WING1|SETTRACKTOLATESTDCA | w1cJoshua | # Sailor Chef -Grimsby US-
WING1|SETTRACKTOLATESTDCA | w1cRay | # Sailor Chef -Triton US-
WING1|SETTRACKTOLATESTDCA | w1cWilliam | # Gull -Eric US-

WING1|SETLATESTDCA|xxx|
WING1|SETTRACKTOLATESTDCA | w1cGabriel | # Eric
WING1|SETTRACKTOLATESTDCA | w1cChadly | # Sebastian
WING1|SETTRACKTOLATESTDCA | w1cAnna | # Flounder
WING1|SETTRACKTOLATESTDCA | w1cMargery | # Ursula
WING1|SETTRACKTOLATESTDCA | w1cJohn | # Triton
WING1|SETTRACKTOLATESTDCA | w1cJudah | # Chef Louis
WING1|SETTRACKTOLATESTDCA | w1cBridget | # Scuttle
WING1|SETTRACKTOLATESTDCA | w1cDylan | # Flotsam
WING1|SETTRACKTOLATESTDCA | w1cElijah | # Jetsam
WING1|SETTRACKTOLATESTDCA | w1cKatelyn | # Aquata Princess 1
WING1|SETTRACKTOLATESTDCA | w1cNadja | # Andrina Princess 2 Gull
WING1|SETTRACKTOLATESTDCA | w1cRachelP | # Arista Princess 3
WING1|SETTRACKTOLATESTDCA | w1cMaura | # Atina Princess 4
WING1|SETTRACKTOLATESTDCA | w1cJacklyn | # Adella Princess 5
WING1|SETTRACKTOLATESTDCA | w1cSarah | # Allana Princess 6 -Ariel US-
WING1|SETTRACKTOLATESTDCA | w1cRebecca | # Maid Gull
WING1|SETTRACKTOLATESTDCA | w1cAnnie | # Maid
WING1|SETTRACKTOLATESTDCA | w1cKara | # Maid
WING1|SETTRACKTOLATESTDCA | w1cGenevieve | # Leeward
WING1|SETTRACKTOLATESTDCA | w1cJocelyn | # Windward
WING1|SETTRACKTOLATESTDCA | w1cRachelS | # -MerSister US-
WING1|SETTRACKTOLATESTDCA | w1cJen | # -Ursula US-
WING1|SETTRACKTOLATESTDCA | w1cPriscilla | # Gull
WING1|SETTRACKTOLATESTDCA | w1cOS1 | # -offstage mic 1
WING1|SETTRACKTOLATESTDCA | w1cOS2 | # -offstage mic 2
WING1|SETTRACKTOLATESTDCA | w1cOS3 | # -offstage mic 3
WING1|SETTRACKTOLATESTDCA | w1cOS4 | # -offstage mic 4

WING1|CUSTOMEndCue|"Cue 140 Done"|

SETLATESTMUTEtoMUTE

SETLATESTMUTEtoMUTE sets the latest mute value to MUTE

Format:
SETLATESTMUTEtoMUTE|

SETLATESTMUTEtoUNMUTE

SETLATESTMUTEtoUNMUTE sets the latest mute value to UNMUTE

Format:
SETLATESTMUTEtoUNMUTE|

SETTRACKTOLATESTMUTE

SETTRACKTOLATESTMUTE sets the specified Track to have the MUTE or UNMUTE value based on whether SETLATESTMUTEtoMUTE or SETKATESTMUTEtoUNMUTE was last read in form the cue file content.

- It sets the mute according to the latest mute value set by either SETLATESTMUTEtoMUTE or SETLATESTMUTEtoUNMUTE

Format:
SETTRACKTOLATESTMUTE|myTrackVar's name|

These 2 commands allow you to just place a SETLATESTMUTEtoMUTE or SETLATESTMUTEtoUNMUTE command in a CUE and then follow it by the SETTRACKTOLATESTMUTE commands for all tracks you want in that condition.

With these you can cut and paste cue text lines under the desired SATELATESTMUTEto… line to manage the cues. This is very efficient and easy to manage.

NOTE: Track color may be changed based on the on and off color values for the channel.

#|

This is the comment indicator
comments begin with #|
Format
#|

X32 CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS

NOTE: Values designated as “myXxx value” are defined in the BUILD files and described above.

NOTE: THIS PORTION OF THE DOCUMENT INDICATES THE CUE COMMAND THAT MUST STILL BE PREFIXED BY THE DEVICE NAME AS SET IN THE BUILD_DEVICE_TYPES.CMD file.

EACH COMMAND MUST BE PRECEEDED BY devicename|
For example:
X1|createfile|

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it calls createfile to open a new output cue file based on the name of the input cue file
2) it renames the main stereo channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value
Format:
CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames Main Stereo Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed
Format:
CUSTOMEndCue|text|

CHAN

CHAN is used to mute or unmute a channel
Format:
CHAN|myChan value|MUTE or UNMUTE

CHANNAME

CHANNAME is used to set a channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
CHANNAME| myChan value|text

CHANCOLOR

CHANCOLOR is used to set the color for a channel in the scribble strip. The Color Names are set in the BUILD script.
Format:
CHANCOLOR| myChan value|Color Name

CHANSEND

CHANSEND is used to set the enable or disable a send from a specified channel.
Format:
CHANSEND|myChan value| myBus value |MUTE or UNMUTE

CHANDCA

CHANDCA is used to set the a Channel into one or more DCAs.
Format:
CHANDCA| myChan value|myDCAassign value|

DCA

DCA is used to mute or unmute a DCA
Format:
DCA|myDCA value|MUTE or UNMUTE

DCANAME

DCANAME is used to set a DCA name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
DCANAME| myDCA value|text

DCACOLOR

DCACOLOR is used to set the color for a DCA in the scribble strip. The Color Names are set in the BUILD script.
Format:
DCACOLOR| myDCA value |myColor value

BUS

BUS is used to mute or unmute a BUS
Format:
BUS|myBus value|MUTE or UNMUTE

BUSNAME

BUSNAME is used to set a BUS name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
BUSNAME| myBus value|text

BUSCOLOR

BUSCOLOR is used to set the color for a BUS in the scribble strip.
Format:
BUSCOLOR|myBus value|myColor value

MAINST

MAINST is used to mute or unmute the Main Stereo Channel
Format:
MAINST|MUTE or UNMUTE

MAINSTNAME

MAINST is used to set a MAIN Stereo Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
MAINSTNAME|text

MAINSTCOLOR

MAINSTCOLOR is used to set the color for the Main Stereo channel in the scribble strip.
Format:
MAINSTCOLOR|myColor value

MAINM

MAINM is used to mute or unmute the Main Mono Channel
Format:
MAINM|MUTE or UNMUTE

MAINMNAME

MAINM is used to set a MAIN Mono Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
MAINMNAME|text

MAINMCOLOR

MAINMCOLOR is used to set the color for the Main Mono channel in the scribble strip.
Format:
MAINMCOLOR|myColor value

SENDand_UNMUTE

SENDand_UNMUTE is used to update BUS SENDS and then UNMUTE the channel.
Format:
SENDand_UNMUTE|myChan value|mySend value|

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update BUS SENDS

Format:
SENDand_MUTE| myChan value|mySend value|

ROUTE_OUT

ROUTE_OUT is used to define a source and target for USER OUT
Format:
ROUTE_OUT|myRouteOut value|myRouteSourceOut value

ROUTE_IN

ROUTE_IN is used to define a source and target for USER IN
Format:
ROUTE_IN| myRouteIn value|myRouteSourceIn value

USB_RECORDER

USB_RECORDER is used control play and record for the USB recorder
Format:
USB_RECORDER|myTape value|

LIVE_RECORDER

LIVE_RECORDER is used control play and record for the LIVE SD recorder
Format:
LIVE_RECORDER|myCard value|

SAVESCENE

SAVESCENE is used to save a scene in slot 0-99 and name it and set a scene note. Enclose strings with spaces in them in quotes.
Format:
SAVESCENE|mySceneNumber|Name|Note|

LOADSCENE

LOADSCENE is used to load a scene from slot 0-99.
Format:
LOADSCENE|mySceneNumber|

SAVESNIPPET

SAVESSNIPPET is used to save a snippet in slot 0-99 and name it. Enclose strings with spaces in them in quotes. Note that the filters used are those already assigned to this snippet slot. Therefore, you can use this to replace content of a snippet but not save a brand new snippet.
Format:
SAVESNIPPET|mySnippetNumber|Name

LOADSNIPPET

LOADSNIPPET is used to load a snippet from slot 0-99.
Format:
LOADSNIPPET|mySnippetNumber|

#|
This is the comment indicator
comments begin with #|
Format
#|

REAPER CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS

NOTE: Values designated as “myXxx value” are defined in the BUILD files and described above.

NOTE: THIS PORTION OF THE DOCUMENT INDICATES THE CUE COMMAND THAT MUST STILL BE PREFIXED BY THE DEVICE NAME AS SET IN THE BUILD_DEVICE_TYPES.CMD file.

EACH COMMAND MUST BE PRECEEDED BY devicename|
For example:
REAPER1|createfile|

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to an existing file by the same name if it exists. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it open a new output cue file based on the name of the input cue file
2) it renames the requested track to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value based on the value specified in the script.
Format:
CUSTOMStartCue|myChan value|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames the requested track to the specified text (perhaps the Cue name) to indicate to you that this is completed
2) it sets up a REAPER action call to 40157 to create a marker on the transport timeline
3) It names that marker by the text specified
Format:
CUSTOMEndCue| myChan value |text|

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS to a volume specified by number and then UNMUTE the channel.
Format:
SENDand_UNMUTE|myChan value|mySend value |number |

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update SENDS to a volume specified by the number
Format:
SENDand_MUTE|myChan value|mySend value|number|

SENDand_ezMUTE

SENDand_ezMUTE is used to mute the channel. The remaining fields are ignored. This command exists so that you can copy and paste commands in cue files and undertake this simple mute without having to change the format of the command. For example, if you are using SENDand_UNMUTE to set sends and then unmute a track, you can just double click on UNMUTE and replace the text with ezMUTE. The track can be unmuted again by just going back and not having to retype the rest of the command.
Format:
SENDand_ezMUTE|myChan value|

SENDand_UNMUTE_NAME

SENDand_UNMUTE_NAME is used to update SENDS and then UNMUTE the channel and set the channel's display to text and volume level to number.
Format:
SENDand_UNMUTE_NAME|myChan value|mySend value|text|number|

SENDand_MUTE_NAME

SENDand_MUTE_NAME is used to mute the channel and then update SENDS to volume in number and set the channel's display name to text.
Format:
SENDand_MUTE_NAME|myChan value|mySend value|text|number|

TRACKPAN

TRACKPAN is used to pan the track. 0 is full left, 1 if full right, .5 is center.
Format:
TRACKPAN|myChan value|pan|

SELECT

SELECT is used to select a track. The Name is optional and if specified renames the track as well as selects it.
Format:
SELECT|myChan value|Name|

UNSELECT

UNSELECT is used to un-select a track. The Name is optional and if specified renames the track as well as unselects it.
Format:
UNSELECT|myChan value|Name|

UNMUTE

UNMUTE is used to UNMUTE the channel and set the channel's display name to text.
Format:
UNMUTE|myChan value|text|

MUTE

MUTE is used to MUTE the channel and set the channel's display name to text.
Format:
MUTE|myChan value|text|

MARKERName

MARKERName to update the name of the specified marker.
Format:
MARKERName|||ID|Name|

LASTMARKERName

LASTMARKERName to update the name of the last marker.
Format:
LASTMARKERName||Name|

GOTOMarker

GOTOMarker is used to go to the nth marker (not Marker number “N” but positionally Nth)
Format:
GOTMarker||number|

RECORD

RECORD is used to toggle start of transport.
Format:
RECORD

PLAY

PLAY is used to toggle play of transport.
Format:
PLAY

STOP

STOP is used to toggle stop of transport.
STOP

PAUSE

PAUSE is used to toggle pause of transport
Format:
PAUSE

FXBYPASS

FXBYPASS is used to bypass an FX for a track
Format:
FXBYPASS|my Chan value|fx number|

FXACTIVE

FXACTIVE is used to activate an FX for a track
Format:
FXACTIVE|my Chan value|fx number|

TRACKNAME

TRACKNAME is used to change the display name of a tracak
Format:
TRACKNAME|my Chan's Name|new display name|

TRACKVOLUME

TRACKVOLUME is used to change the volume of a track to number
Format:
TRACKVOLUME|my Chan value|number|

ACTIONi

ACTIONi takes an action number as an argument and sends it
A handy action number to keep in mind is 41743 which sends a refresh to subscribed interfacing systems. This does nothing for these scripts since they are listen only, but will be useful to refresh other devices such as TOUCHOSC interfaces. An action list can be found at https://forum.cockos.com/showthread.php?t=23236
Format:
ACTIONi||action number|

ACTIONs

ACTIONs takes an action text string as an argument and sends it
An action list can be found at https://forum.cockos.com/showthread.php?t=23236
Format:
ACTIONs||action number|

ARMON

ARMON is used to ARM the channel and set the channel's display name
ARMON|myChan's Name|Channel Display Name|

ARMOFF

ARMOFF is used to UN-ARM the channel and set the channel's display name
ARMOFF|myChan's Name|Channel Display Name|

AUTOMIXON

AUTOMIXON is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 1 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.
AUTOMIXON|myChan's Name|

AUTOMIXOFF

AUTOMIXOFF is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 0 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.

AUTOMIXON|myChan's Name|

PLUGIN_ADJ

PLUGIN_ADJ updates plugin settings based on a myPlugin pattern string and custom code (see details above).
PLUGIN_ADJ|myChan's Name|myPlugin's Name|optional Variable Value|optional Variable Value|

FXOPENUI

FXOPENUI is used to open an FX for a track
Format:
FXOPENUI|my Chan value|fx number|

FXCLOSEUI

FXCLOSEUI is used to close an FX for a track
Format:
FXCLOSEUI|my Chan value|fx number|

#|

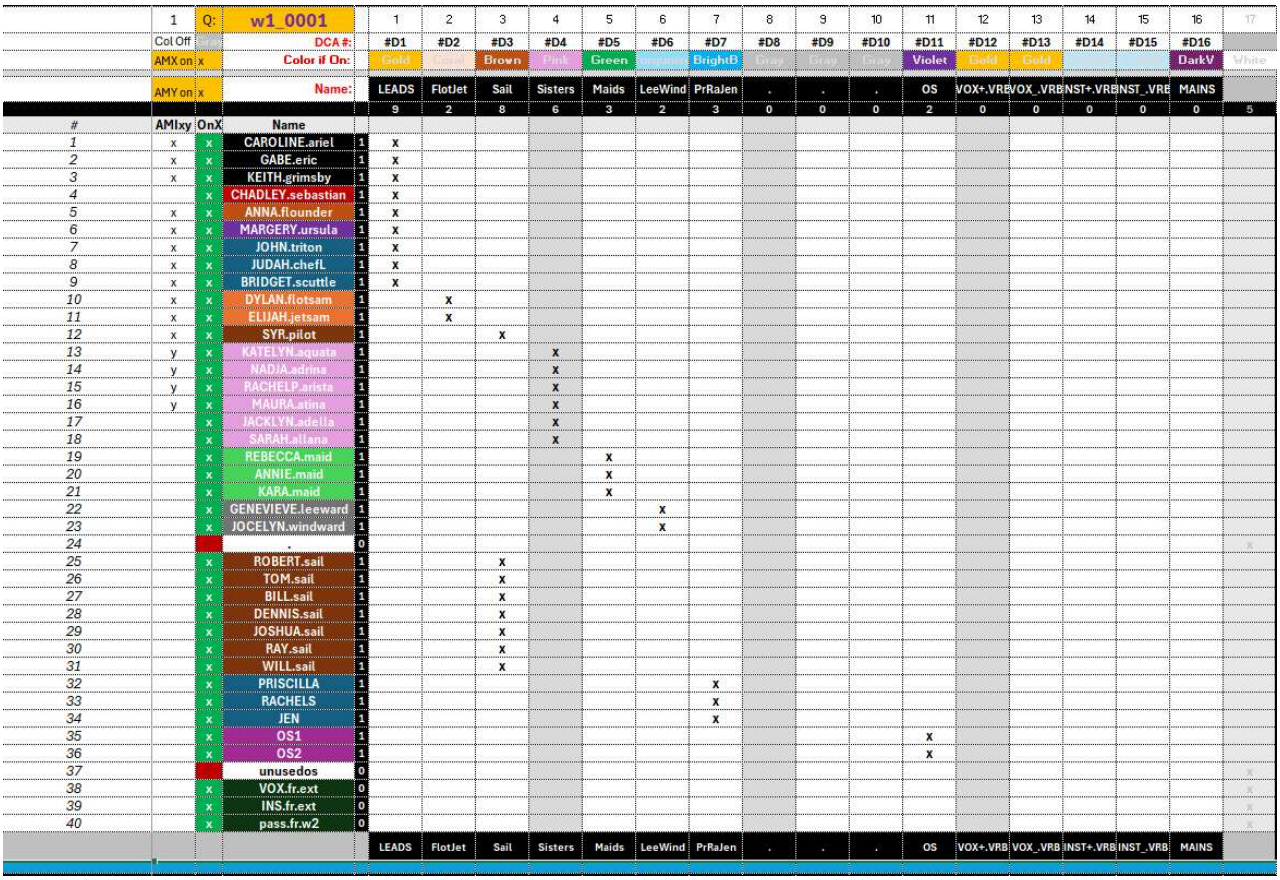
This is the comment indicator
comments begin with #|
Format
#|

VARIABLEFROMFILE Cue Content Generators

The VARIABLEFROMFILE cue entries are described above and are used to inject externally written OSC content directly into Cues.

Provided such tools are described here.

VARIABLEFROMFILE_40chans_1 version 20261026b Excel Tool



	1	Q:	w1_0001	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	Col Off		DCA #:	#D1	#D2	#D3	#D4	#D5	#D6	#D7	#D8	#D9	#D10	#D11	#D12	#D13	#D14	#D15	#D16	
	AMX on	x	Color if On:	Gold	Yellow	Brown	Pink	Green	Aqua	BrightB	Grey	Grey	Grey	Violet	Gold	Gold			DarkV	White
	AMY on	x	Name:	LEADS	FlotJet	Sail	Sisters	Maids	LeeWind	PrRaJen	.	.	.	OS	VOX+.VRB	VOX_.VRB	INST+.VRB	INST_.VRB	MAINS	
				9	2	8	6	3	2	3	0	0	0	2	0	0	0	0	0	5
#	AMIxy	OnX	Name																	
1	x	x	CAROLINE.ariel	1	x															
2	x	x	GABE.eric	1	x															
3	x	x	KEITH.grimsby	1	x															
4		x	CHADLEY.sebastian	1	x															
5	x	x	ANNA.flounder	1	x															
6	x	x	MARGERY.ursula	1	x															
7	x	x	JOHN.triton	1	x															
8	x	x	JUDAH.chefL	1	x															
9	x	x	BRIDGET.scuttle	1	x															
10	x	x	DYLAN.flotsam	1		x														
11	x	x	ELIJAH.jetsam	1		x														
12	x	x	SYR.pilot	1			x													
13	y	x	KATELYN.arista	1				x												
14	y	x	NADIA.adrina	1				x												
15	y	x	RACHELP.arista	1				x												
16	y	x	MAURA.ariana	1				x												
17		x	JACKLYN.adella	1				x												
18		x	SARAH.allana	1				x												
19		x	REBECCA.maid	1					x											
20		x	ANNIE.maid	1					x											
21		x	KARA.maid	1					x											
22		x	GENEVIEVE.leeward	1						x										
23		x	JOCELYN.windward	1						x										
24			-	0																x
25		x	ROBERT.sail	1			x													
26		x	TOM.sail	1			x													
27		x	BILL.sail	1			x													
28		x	DENNIS.sail	1			x													
29		x	JOSHUA.sail	1			x													
30		x	RAY.sail	1			x													
31		x	WILL.sail	1			x													
32		x	PRISCILLA	1						x										
33		x	RACHELS	1						x										
34		x	JEN	1						x										
35		x	OS1	1										x						
36		x	OS2	1										x						
37			unusedos	0																x
38		x	VOX.fr.ext	0																x
39		x	INS.fr.ext	0																x
40		x	pass.fr.w2	0																x
				LEADS	FlotJet	Sail	Sisters	Maids	LeeWind	PrRaJen	.	.	.	OS	VOX+.VRB	VOX_.VRB	INST+.VRB	INST_.VRB	MAINS	

This Excel sheet includes formulas to update mute status, names, DCA assignments, and Automix assignments and status along with colors for the 40 WING channels. It does this very efficiently through use of the node syntax for OSC for the WING. Everything specified gets updated seemingly instantly.

The README tab in the excel file has a complete description of this tool and its use.

The tool creates 7 variables of content for each cue named as follows in the provided script:

w1_0001_ch_col_mute
w1_0001_ch_tags
w1_0001_ch_names_a
w1_0001_ch_names_b
w1_0001_ch_postins_a
w1_0001_ch_postins_b

w1_0001_dcas_amix

An Example of use in a Cue would be through lines such as these:

WING1|VARIABLEFROMFILE|w1_0001_ch_col_mute|

WING1|VARIABLEFROMFILE|w1_0001_ch_tags|
WING1|VARIABLEFROMFILE|w1_0001_ch_names_a|
WING1|VARIABLEFROMFILE|w1_0001_ch_names_b|

WING1|VARIABLEFROMFILE|w1_0001_ch_postins_a|

WING1|VARIABLEFROMFILE|w1_0001_ch_postins_b|

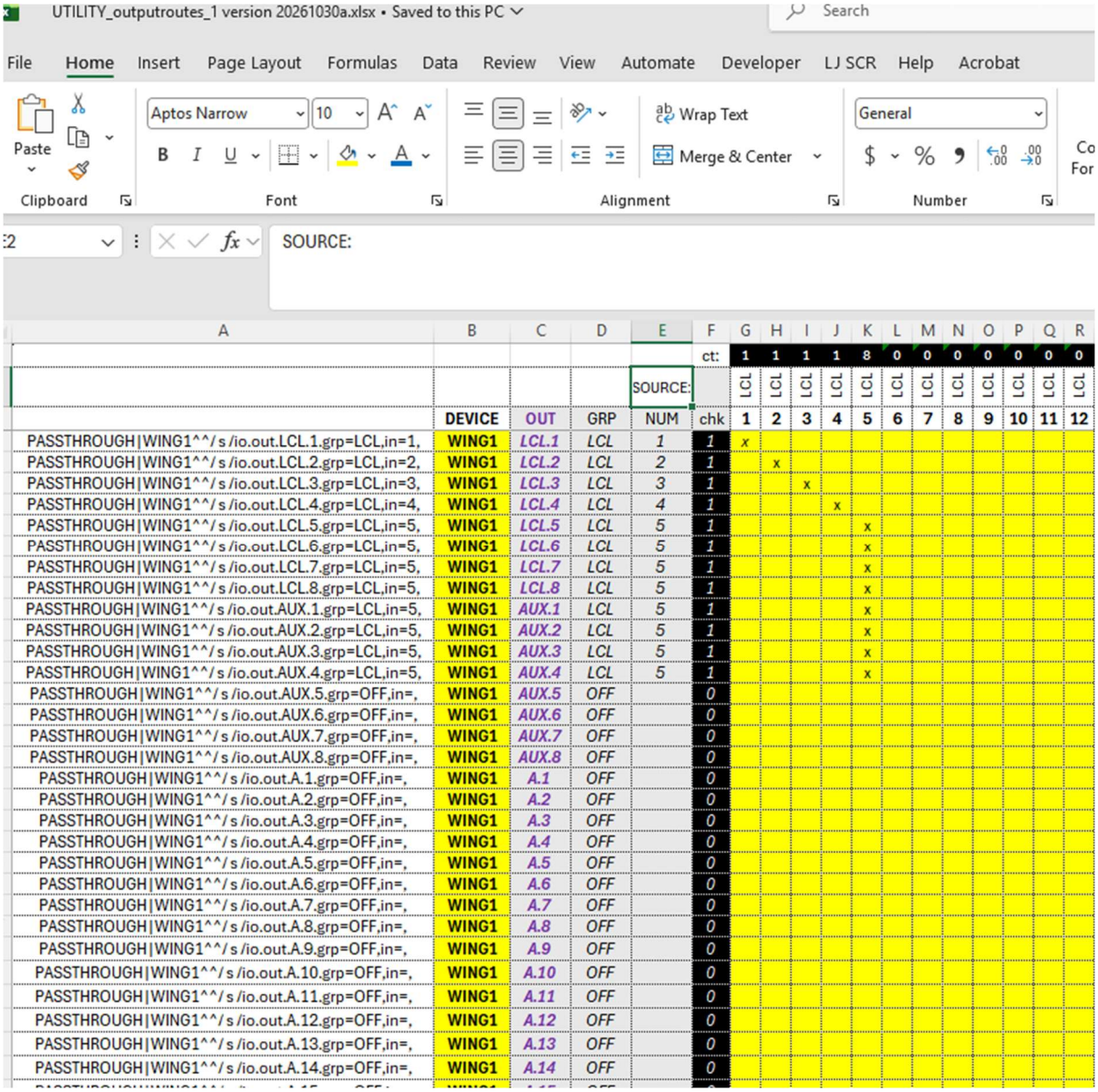
WING1|VARIABLEFROMFILE|w1_0001_dcas_amix|

WING UTILITY Cue Content Generators to COPY and PASTE into Cue Files

These are Excel documents that you can use to set up information for the WING. TO insert the OSC into a CUE simply copy the cue content column fomr the excel file and paste it into a Cue that you can then run through the Cue system.

UTILITY_outputroutes_1 version 20261030a.xlsx

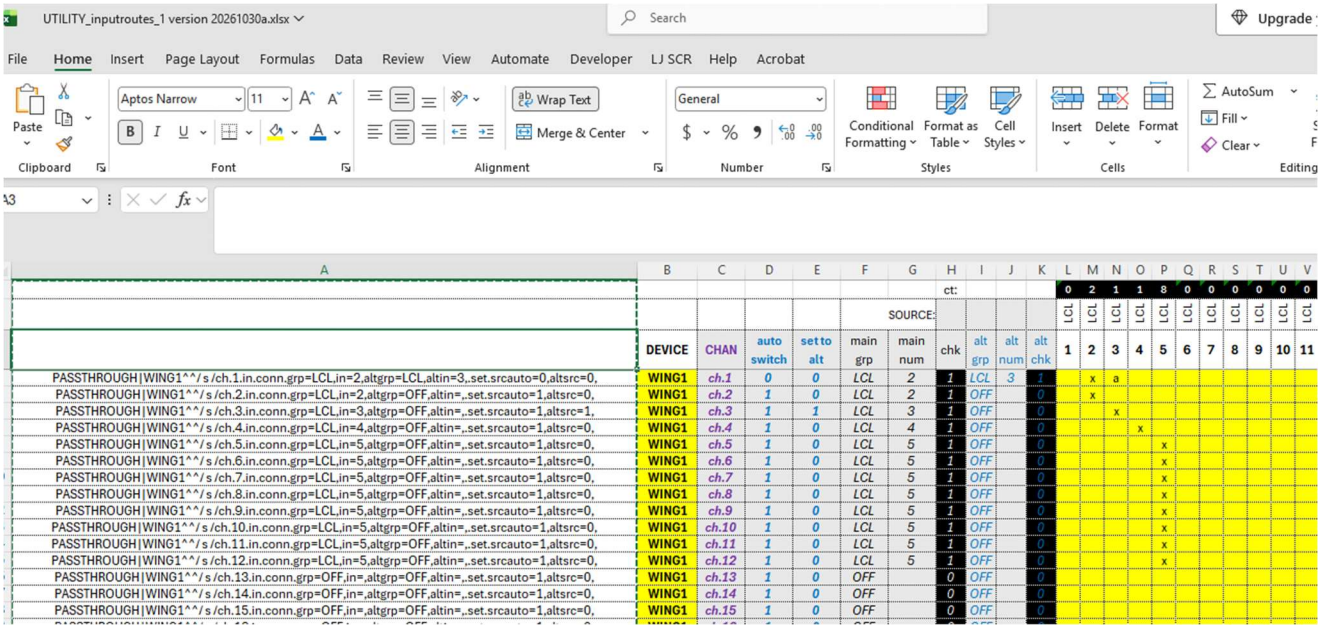
This excel document can be used to set up cue content for output routing for the WING via a grid-based spreadsheet. See the README tab in the file for details.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
						ct:	1	1	1	1	8	0	0	0	0	0	0	0
					SOURCE:		LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL
		DEVICE	OUT	GRP	NUM	chk	1	2	3	4	5	6	7	8	9	10	11	12
	PASSTHROUGH|WING1^^/s/io.out.LCL.1.grp=LCL,in=1,	WING1	LCL.1	LCL	1	1	x											
	PASSTHROUGH|WING1^^/s/io.out.LCL.2.grp=LCL,in=2,	WING1	LCL.2	LCL	2	1		x										
	PASSTHROUGH|WING1^^/s/io.out.LCL.3.grp=LCL,in=3,	WING1	LCL.3	LCL	3	1			x									
	PASSTHROUGH|WING1^^/s/io.out.LCL.4.grp=LCL,in=4,	WING1	LCL.4	LCL	4	1				x								
	PASSTHROUGH|WING1^^/s/io.out.LCL.5.grp=LCL,in=5,	WING1	LCL.5	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.LCL.6.grp=LCL,in=5,	WING1	LCL.6	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.LCL.7.grp=LCL,in=5,	WING1	LCL.7	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.LCL.8.grp=LCL,in=5,	WING1	LCL.8	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.AUX.1.grp=LCL,in=5,	WING1	AUX.1	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.AUX.2.grp=LCL,in=5,	WING1	AUX.2	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.AUX.3.grp=LCL,in=5,	WING1	AUX.3	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.AUX.4.grp=LCL,in=5,	WING1	AUX.4	LCL	5	1					x							
	PASSTHROUGH|WING1^^/s/io.out.AUX.5.grp=OFF,in=,	WING1	AUX.5	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.AUX.6.grp=OFF,in=,	WING1	AUX.6	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.AUX.7.grp=OFF,in=,	WING1	AUX.7	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.AUX.8.grp=OFF,in=,	WING1	AUX.8	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.1.grp=OFF,in=,	WING1	A.1	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.2.grp=OFF,in=,	WING1	A.2	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.3.grp=OFF,in=,	WING1	A.3	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.4.grp=OFF,in=,	WING1	A.4	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.5.grp=OFF,in=,	WING1	A.5	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.6.grp=OFF,in=,	WING1	A.6	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.7.grp=OFF,in=,	WING1	A.7	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.8.grp=OFF,in=,	WING1	A.8	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.9.grp=OFF,in=,	WING1	A.9	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.10.grp=OFF,in=,	WING1	A.10	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.11.grp=OFF,in=,	WING1	A.11	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.12.grp=OFF,in=,	WING1	A.12	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.13.grp=OFF,in=,	WING1	A.13	OFF		0												
	PASSTHROUGH|WING1^^/s/io.out.A.14.grp=OFF,in=,	WING1	A.14	OFF		0												

UTILITY_inputroutes_1 version 20261030a.xlsx

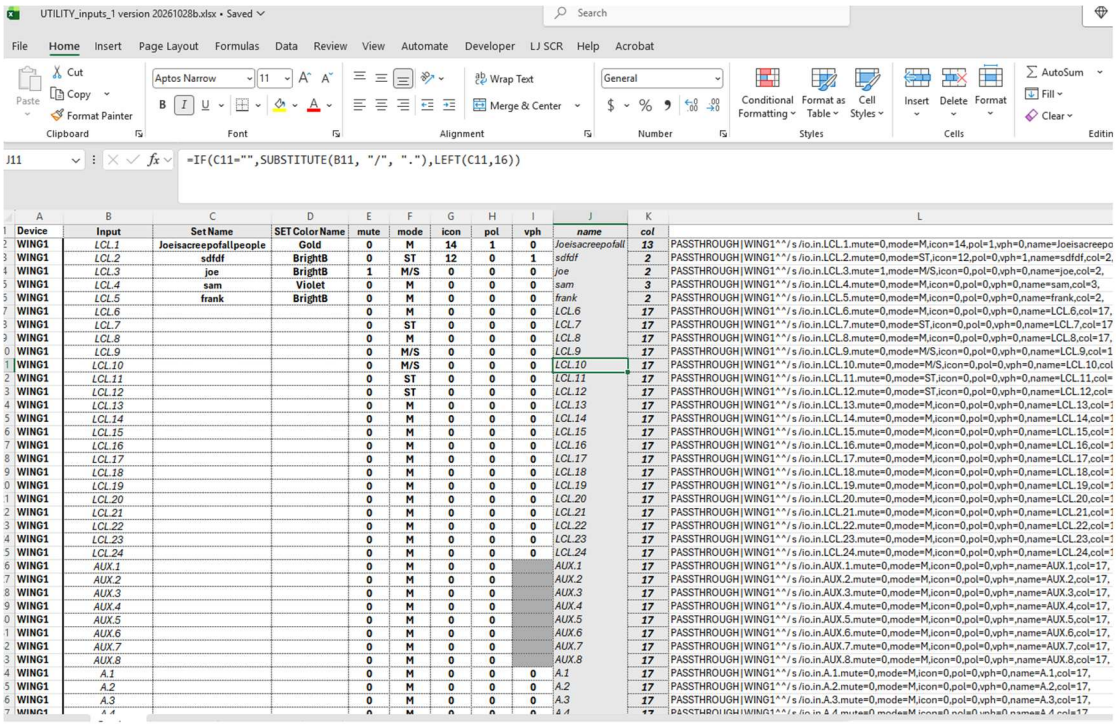
This excel document can be used to set up cue content for input routing for the WING via a grid-based spreadsheet. Main and Alt sources can be selected from a grid and the individual/global and initial source choice are able to be set. See the README tab in the file for details.



A	DEVICE	CHAN	auto switch	set to alt	main grp	main num	chk	alt grp	alt num	alt chk	1	2	3	4	5	6	7	8	9	10	11
						ct:					0	2	1	1	8	0	0	0	0	0	0
						SOURCE:					LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL	LCL
PASSTHROUGH|WING1^^/s/ch.1.in.conn.grp=LCL,in=2,altgrp=LCL,altin=3,.set.srcauto=0,altsrc=0,	WING1	ch.1	0	0	LCL	2	1	LCL	3	1		x	a								
PASSTHROUGH|WING1^^/s/ch.2.in.conn.grp=LCL,in=2,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.2	1	0	LCL	2	1	OFF		0		x									
PASSTHROUGH|WING1^^/s/ch.3.in.conn.grp=LCL,in=3,altgrp=OFF,altin=,.set.srcauto=1,altsrc=1,	WING1	ch.3	1	1	LCL	3	1	OFF		0			x								
PASSTHROUGH|WING1^^/s/ch.4.in.conn.grp=LCL,in=4,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.4	1	0	LCL	4	1	OFF		0				x							
PASSTHROUGH|WING1^^/s/ch.5.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.5	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.6.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.6	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.7.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.7	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.8.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.8	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.9.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.9	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.10.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.10	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.11.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.11	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.12.in.conn.grp=LCL,in=5,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.12	1	0	LCL	5	1	OFF		0					x						
PASSTHROUGH|WING1^^/s/ch.13.in.conn.grp=OFF,in=,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.13	1	0	OFF		0	OFF		0											
PASSTHROUGH|WING1^^/s/ch.14.in.conn.grp=OFF,in=,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.14	1	0	OFF		0	OFF		0											
PASSTHROUGH|WING1^^/s/ch.15.in.conn.grp=OFF,in=,altgrp=OFF,altin=,.set.srcauto=1,altsrc=0,	WING1	ch.15	1	0	OFF		0	OFF		0											

UTILITY_ inputs_1 version 20261028b.xlsx

This excel document can be used to set up cue content to allow naming and color coding and icon setting, etc. for WING Inputs. See the README tab in the file for details.



Device	Input	Set Name	SET Color Name	mute	mode	icon	pol	vph	name	col	L
WING1	LCL.1	Joeisacreepofallpeople	Gold	0	M	14	1	0	Joeisacreepofall	13	PASSTHROUGH|WING1^^/s/io.in.LCL.1.mute=0,mode=M,icon=14,pol=1,vph=0,name=Joeisacreepo
WING1	LCL.2	sdfdf	BrightB	0	ST	12	0	1	sdfdf	2	PASSTHROUGH|WING1^^/s/io.in.LCL.2.mute=0,mode=ST,icon=12,pol=0,vph=1,name=sdfdf,col=2,
WING1	LCL.3	joe	BrightB	1	M/S	0	0	0	joe	2	PASSTHROUGH|WING1^^/s/io.in.LCL.3.mute=1,mode=M/S,icon=0,pol=0,vph=0,name=joe,col=2,
WING1	LCL.4	sam	Violet	0	M	0	0	0	sam	3	PASSTHROUGH|WING1^^/s/io.in.LCL.4.mute=0,mode=M,icon=0,pol=0,vph=0,name=sam,col=3,
WING1	LCL.5	frank	BrightB	0	M	0	0	0	frank	2	PASSTHROUGH|WING1^^/s/io.in.LCL.5.mute=0,mode=M,icon=0,pol=0,vph=0,name=frank,col=2,
WING1	LCL.6			0	M	0	0	0	LCL.6	17	PASSTHROUGH|WING1^^/s/io.in.LCL.6.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.6,col=17,
WING1	LCL.7			0	ST	0	0	0	LCL.7	17	PASSTHROUGH|WING1^^/s/io.in.LCL.7.mute=0,mode=ST,icon=0,pol=0,vph=0,name=LCL.7,col=17
WING1	LCL.8			0	M	0	0	0	LCL.8	17	PASSTHROUGH|WING1^^/s/io.in.LCL.8.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.8,col=17,
WING1	LCL.9			0	M/S	0	0	0	LCL.9	17	PASSTHROUGH|WING1^^/s/io.in.LCL.9.mute=0,mode=M/S,icon=0,pol=0,vph=0,name=LCL.9,col=1
WING1	LCL.10			0	M/S	0	0	0	LCL.10	17	PASSTHROUGH|WING1^^/s/io.in.LCL.10.mute=0,mode=M/S,icon=0,pol=0,vph=0,name=LCL.10,col
WING1	LCL.11			0	ST	0	0	0	LCL.11	17	PASSTHROUGH|WING1^^/s/io.in.LCL.11.mute=0,mode=ST,icon=0,pol=0,vph=0,name=LCL.11,col=
WING1	LCL.12			0	ST	0	0	0	LCL.12	17	PASSTHROUGH|WING1^^/s/io.in.LCL.12.mute=0,mode=ST,icon=0,pol=0,vph=0,name=LCL.12,col=
WING1	LCL.13			0	M	0	0	0	LCL.13	17	PASSTHROUGH|WING1^^/s/io.in.LCL.13.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.13,col=1
WING1	LCL.14			0	M	0	0	0	LCL.14	17	PASSTHROUGH|WING1^^/s/io.in.LCL.14.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.14,col=1
WING1	LCL.15			0	M	0	0	0	LCL.15	17	PASSTHROUGH|WING1^^/s/io.in.LCL.15.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.15,col=1
WING1	LCL.16			0	M	0	0	0	LCL.16	17	PASSTHROUGH|WING1^^/s/io.in.LCL.16.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.16,col=1
WING1	LCL.17			0	M	0	0	0	LCL.17	17	PASSTHROUGH|WING1^^/s/io.in.LCL.17.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.17,col=1
WING1	LCL.18			0	M	0	0	0	LCL.18	17	PASSTHROUGH|WING1^^/s/io.in.LCL.18.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.18,col=1
WING1	LCL.19			0	M	0	0	0	LCL.19	17	PASSTHROUGH|WING1^^/s/io.in.LCL.19.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.19,col=1
WING1	LCL.20			0	M	0	0	0	LCL.20	17	PASSTHROUGH|WING1^^/s/io.in.LCL.20.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.20,col=1
WING1	LCL.21			0	M	0	0	0	LCL.21	17	PASSTHROUGH|WING1^^/s/io.in.LCL.21.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.21,col=1
WING1	LCL.22			0	M	0	0	0	LCL.22	17	PASSTHROUGH|WING1^^/s/io.in.LCL.22.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.22,col=1
WING1	LCL.23			0	M	0	0	0	LCL.23	17	PASSTHROUGH|WING1^^/s/io.in.LCL.23.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.23,col=1
WING1	LCL.24			0	M	0	0	0	LCL.24	17	PASSTHROUGH|WING1^^/s/io.in.LCL.24.mute=0,mode=M,icon=0,pol=0,vph=0,name=LCL.24,col=1
WING1	AUX.1			0	M	0	0		AUX.1	17	PASSTHROUGH|WING1^^/s/io.in.AUX.1.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.1,col=17,
WING1	AUX.2			0	M	0	0		AUX.2	17	PASSTHROUGH|WING1^^/s/io.in.AUX.2.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.2,col=17,
WING1	AUX.3			0	M	0	0		AUX.3	17	PASSTHROUGH|WING1^^/s/io.in.AUX.3.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.3,col=17,
WING1	AUX.4			0	M	0	0		AUX.4	17	PASSTHROUGH|WING1^^/s/io.in.AUX.4.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.4,col=17,
WING1	AUX.5			0	M	0	0		AUX.5	17	PASSTHROUGH|WING1^^/s/io.in.AUX.5.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.5,col=17,
WING1	AUX.6			0	M	0	0		AUX.6	17	PASSTHROUGH|WING1^^/s/io.in.AUX.6.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.6,col=17,
WING1	AUX.7			0	M	0	0		AUX.7	17	PASSTHROUGH|WING1^^/s/io.in.AUX.7.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.7,col=17,
WING1	AUX.8			0	M	0	0		AUX.8	17	PASSTHROUGH|WING1^^/s/io.in.AUX.8.mute=0,mode=M,icon=0,pol=0,vph=,name=AUX.8,col=17,
WING1	A.1			0	M	0	0	0	A.1	17	PASSTHROUGH|WING1^^/s/io.in.A.1.mute=0,mode=M,icon=0,pol=0,vph=0,name=A.1,col=17,
WING1	A.2			0	M	0	0	0	A.2	17	PASSTHROUGH|WING1^^/s/io.in.A.2.mute=0,mode=M,icon=0,pol=0,vph=0,name=A.2,col=17,
WING1	A.3			0	M	0	0	0	A.3	17	PASSTHROUGH|WING1^^/s/io.in.A.3.mute=0,mode=M,icon=0,pol=0,vph=0,name=A.3,col=17,

Other Tooling

Mixing Station 128 Head Amp Control for X32/M32

I have developed a Mixing Station layout I use to manage gains and Phantom Power for the 128 inputs that can have Head Amps in an x/m32 configuration (Local 1-32 + AES50 A 1-48 + AES50B 1-48 is 128 inputs). This is shared in the Mixing Station Community and is also included in this package as HAs….msz. The latest revision to Mixing Station seriously impacted colors but it still functions.

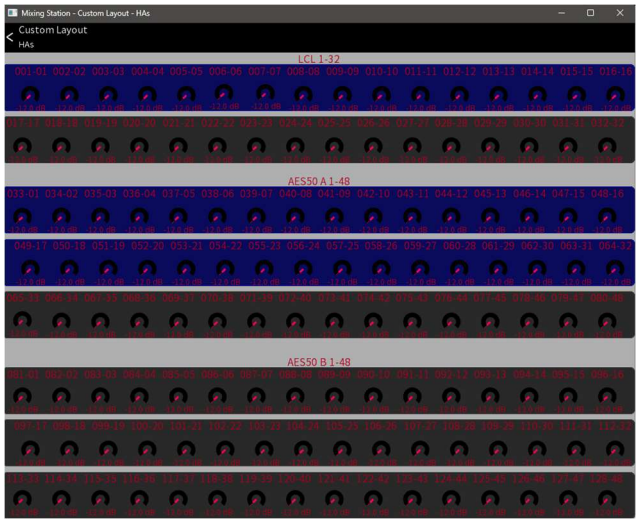


Figure 2 Mixing station x32/m32 Head Amp control for all 128 connections

WING 40 Channel Solo control

I have developed a Mixing Station layout I use to solos for a mic technician. This presents the 40 WING channels along with current mute status, a level meter to help isolate problem mics, channel names, and solo buttons. This allows a remote tech to solo channels and fix mic problems without requiring as much assistance from the main audio mixer. The headphone out is routed back into an inputon the wing and presented remotely for a headphone connection. I use an old analogue mixer and an direct out going to a local or AES50 output. This is shared as SOLO….msz.



Figure 3 Mixing station WING solo management

AutoHotKey Script

AutoHotKey ScriptIf you want hotkeys to run utility cues in this cue system, it is recommended that you try AutoHotKey software. A sample AHT file is included in this package.

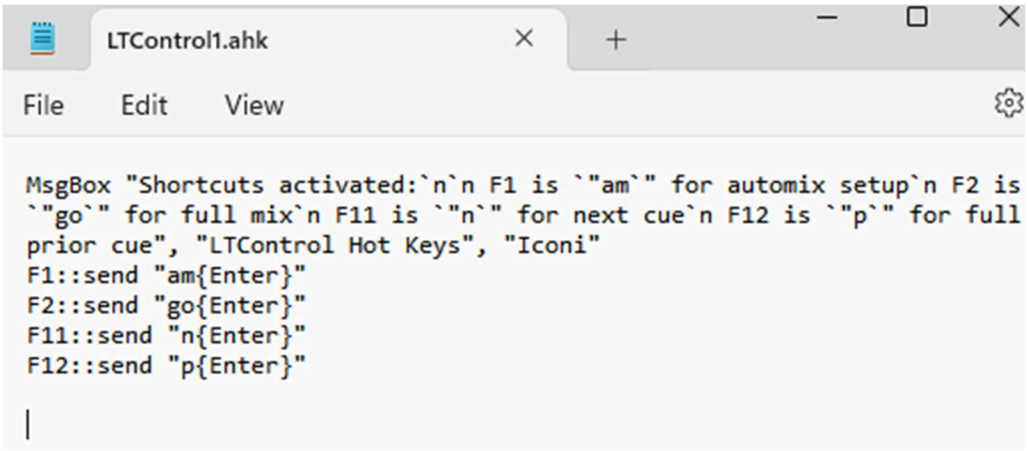


Figure 4 AutoHotKey sample script

Bitfocus Companion and Vicreo Listener

Bitfocus Companion and the Vicreo Listener can be used to provide a web interface to control the Cues. Run the Companion link with /tablet to see the simulated buttons.

Links

Download of **this document**, the **Track Muter, Track Muter and Volume setting** Web interface for REAPER, **Cue System and OSC command system**: https://workwebs.com

Facebook page for the Ecosystem in general and the WorkWebs tools: https://www.facebook.com/profile.php?id=61557113121231

TheatreMix by James Holt: https://theatremix.com/

Open Stage Control OSC interface builder/tool: https://openstagecontrol.ammd.net/

Fully Kiosk Browser for Android: https://www.fully-kiosk.com/

Paul Vannatto's OSC file processing tool: https://sourceforge.net/projects/lt-command/

AutoHotKey software for Windows: https://www.autohotkey.com/

SSL Solid State Logic plugin link: https://solidstatelogic.com/products/ssl-complete-bundle-subscription

WT Automixer for 16 tracks of Automixing in REAPER is available at: https://www.wtautomixer.com/

SSL 360 is available from the Support->Controllers link: https://support.solidstatelogic.com/hc/en-gb

REAPER: https://www.reaper.fm/download.php

DN9630 information: https://klarkteknik.com/product.html?modelCode=0606-ACT

General Behringer x32 information: https://www.behringer.com/catalog.html

Notepad++ powerful text editor for use on Windows: https://notepad-plus-plus.org/

Mixing Station: https://mixingstation.app/

Vicreo Listener: https://www.vicreo-listener.com/

Bitfocus Companion: https://bitfocus.io/companion

WING wosc and link to current OSC command list: https://sites.google.com/site/patrickmaillot/wing

VBAudio Matrix Coconut: To combine multiple ASIO compatible (and other) inputs together into a virtual interface under Windows: https://vb-audio.com/Matrix/coconut.htm